```
class SessionHandler implements SessionHandlerInterface,
SessionIdInterface {
    /* Methods */
    public close(): bool
    public create_sid(): string
    public destroy(string $id): bool
    public gc(int $max_lifetime): int|false
    public open(string $path, string $name): bool
    public read(string $id): string|false
    public write(string $id, string $data): bool
}
```

I say "somewhat" because this class isn't intended to be used directly; you wouldn't instantiate it yourself and call the methods directly. Only the functional API is intended for consumption by developers. The `SessionHandler` class is still useful though, because an instance of this class is used internally by the session functions (e.g., calling `session_start()` will result in a call to `SessionHandler::open()`), and this internal object can be replaced. The `session_set_save_handler()` function can be passed a new instance of `SessionHandler`, which replaces the default handler with the new one. In this way you can inject your own implementation(s) by extending the `SessionHandler` class and overloading one or more of the handler methods:

```
class ModifiedSessionHandler extends SessionHandler
{
    public function read(string $id): string
    {
        $data = parent::read();
        // new code goes here...
        return $data;
    }
    // other overloaded methods as required...
}

// do this before session_start()!
session_set_save_handler(new ModifiedSessionHandler());
session_start();
```